

CHAKRAVYUH

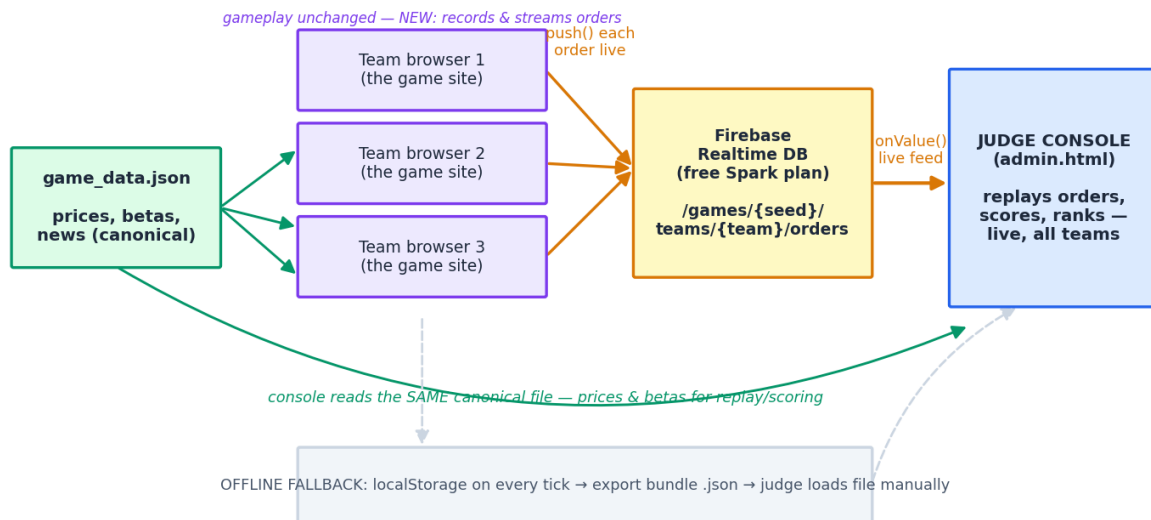
Judge Console & Scoring

Full Implementation Handover (v2)

The three judging criteria, the exact maths, the backend, the code, and how it all wires together.

How the judging layer fits around the game

Teams play exactly as before. Every order is also streamed to a free cloud store; the judge console watches that store live.



Version: v2 — supersedes the v1 design note. Adds the full backend (Firebase, free tier), live scoring, client recording, and complete code guides.

Companion to: Chakravyuh Trading Game — Developer Handover (v5). Nothing there changes; this document adds the judging layer around it.

Audience: developer (Parts 1, 2, 5, 6) · judges (Part 3) · model owner / maths (Parts 4, 7).

How to read this document

This one document contains everything needed to build and operate the judging layer. It is written for three readers, and each part is labelled so you can jump to what you need:

- **The developer** — read Part 1 (architecture), Part 2 (what the console looks like), Part 5 (the code, file by file, with implementations of every calculation), and Part 6 (Firebase setup, exact touchpoints in the existing app.js, and the event-day runbook). You do not need to understand WHY the maths is what it is — Part 5 gives you WHAT to compute, line by line. When something looks arbitrary, Part 4 has the reason.
- **The judges** — read Part 3 only. It says what the three numbers mean, why each is rewarded, how much each is worth, and what the sliders do.
- **The model owner / anyone auditing the maths** — Part 4 is the exact specification of every calculation (it is the glue: the code in Part 5 implements Part 4 equation by equation), and Part 7 is the tuning table.

Table of contents

Right-click here and choose “Update Field” to build the contents.

The single most important idea

The judge console is a PURE FUNCTION: $\text{score} = f(\text{orders}, \text{game_data.json}, \text{weights})$. Teams' orders stream into a free cloud store as they trade; the console replays those raw orders against the canonical price file, computes three subscores per team, blends them with judge-set weights, and ranks. It trusts NOTHING the client computed — every number on screen can be traced back to individual trades, by anyone, after the event. This mirrors the game's own design principle: pre-computed data in, deterministic playback out.

Part 1 — The big picture

1.1 What is being added, in one paragraph

The game itself is untouched: teams still log in, watch news, and trade against the pre-generated price file. Three things are added around it: (1) the game client starts RECORDING — every order and an equity snapshot per tick — and streams orders to a free Firebase Realtime Database as they happen; (2) a new page, `admin.html`, subscribes to that database and shows judges a LIVE leaderboard scored out of 100 on three criteria; (3) a small pure-maths library, `scoring.js`, does all the calculating, so the scoring is testable and auditable in isolation.

How the judging layer fits around the game

Teams play exactly as before. Every order is also streamed to a free cloud store; the judge console watches that store live.

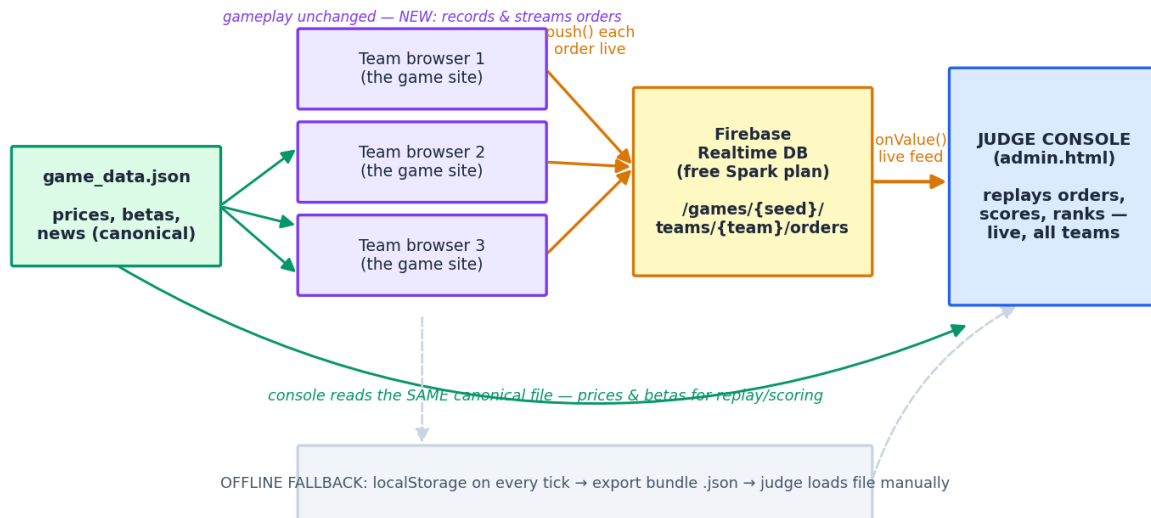


Figure. Teams play as before; orders stream to a free cloud store; the console watches it live. The dashed path is the no-wifi fallback.

1.2 The four moving parts

- **The game client (app.js, existing)** — gains a recorder: push each confirmed order to Firebase AND mirror everything to localStorage (crash recovery). ~60 lines added, zero gameplay change. Part 6.1 lists the exact functions to touch.
- **Firebase Realtime Database (new, free)** — the mailbox. A JSON tree keyed by game seed and team; teams append orders, the console listens. Free Spark plan: no card, 1 GB storage, 100 simultaneous connections — a 10-team event uses well under 1% of that. Part 6.2 has setup + rules.
- **scoring.js (new)** — pure functions, no DOM, no network. Orders in → subscores out. Every function implements one equation from Part 4 and cites it in a comment.
- **admin.html + admin.js (new)** — the judges' page. Subscribes to Firebase, throttles recomputes, draws the leaderboard, weight sliders, and per-team drill-downs.

1.3 Why live scoring is safe to show judges

Because scoring is deterministic replay, a mid-game score is simply "the score if the game ended now": open positions are marked to market, hit rate counts force-closed trips, exposures are averaged over ticks so far. Nothing about the final score depends on having watched it live — reloading the console after the event reproduces the identical final board from the stored orders.

Why recompute from orders instead of trusting the client's numbers?

The client's own portfolio math runs in the players' browsers, where anyone can open devtools. The console therefore replays raw orders against the canonical `game_data.json` prices and rebuilds cash, holdings and equity itself. The client's submitted equity curve is kept only as a cross-check: if it disagrees with the replay beyond rounding, the row is flagged for review (bug, stale file, or tampering) instead of silently scoring. In a group project this is the transparency guarantee.

1.4 Deliberately out of scope in v2

- **No accounts/auth.** Team identity is the same trusted team-name login the game already uses. Firebase rules validate the SHAPE of writes, not identity (Part 6.2 discusses the accepted risk and the cheap mitigations).
- **No drawdown penalties or activity bonuses.** Three criteria only. Add later if events show the need.
- **No changes to `market_engine.py` or `game_data.json`.** The scoring layer reads the file; it never writes it.

Part 2 — Frontend design (for the developer)

One page, four zones, top to bottom. An interactive HTML mockup accompanies this handover; the wireframe below is the same layout.



Figure. The judge console: top bar, weights strip, live leaderboard, and per-team drill-down.

2.1 Top bar

Brand + "Judge Console"; a scope selector (Both sessions / Session 1 / Session 2 — scoring runs on the chosen slice); a live status pill (● LIVE — n teams, or OFFLINE with a file-picker to load exported bundles); Export CSV.

2.2 Weights strip

Three sliders — Return, Diversification, Hit rate — defaulting to 45 / 25 / 30, each showing its value. Moving a slider re-blends and re-sorts instantly. Subscores are NOT recomputed on slider moves (Part 5.5): the sliders are a values dial, not a maths dial, and the UI copy should say so.

2.3 Leaderboard

One row per team: rank, team name, total /100 (large, monospace), then three subscore cells — each showing the human-readable raw stat (+14.7%, 4.3/5 factors, 6/8 hits) above a 0–100 fill bar. Sorted by total, descending. Flagged teams (failed cross-check, insufficient activity) show a warning chip instead of silently ranking.

2.4 Team drill-down (click a row)

- **Equity vs market** — replayed equity curve vs the equal-weight market index (same definition as the game's Portfolio tab), final return and alpha in the header.
- **Factor exposure** — a 10-axis radar of time-averaged |exposure| per factor — the diversification story at a glance (spiky one-axis blob vs spread web) — with effective rank and neutrality v printed beside it, plus a hit-rate donut.
- **Round-trip blotter** — every trip: symbol, side, entry→exit, trip return %, HIT/MISS chip, and whether it was force-closed at the bell. This is the audit trail.

2.5 Live behaviour

New orders arriving over Firebase mark the affected team dirty; a 2-second throttle rescores dirty teams and re-renders. A subtle flash on a row that changed rank keeps judges oriented. If the connection drops, the status pill flips to OFFLINE and the console keeps the last data — scores never silently go stale without the pill saying so.

Part 3 — For the judges: what is rewarded, and why

Every team starts with ₹10,00,000 and trades the same market. They are scored out of 100 on three criteria. The philosophy: reward PROFIT, but also reward the CRAFT that distinguishes a trader from a gambler — independent, hedged bets and a good strike rate.

3.1 Return — 45% of the score

What it is: percentage growth of the team's total portfolio (cash + holdings) from start to end.

Why it matters: it is the game's objective. A trading contest that doesn't chiefly reward P&L teaches the wrong lesson.

Why 45 and not 100: raw P&L over 40 minutes contains real luck. One team riding a single lucky headline can out-earn a disciplined team — the other two criteria exist to separate those cases. Return is deliberately the largest single weight, but not a majority on its own.

How it becomes points: the return percentage passes through a gentle S-curve — roughly, 0% return scores in the 30s, +12% in the 70s, and beyond +20% it flattens near the top. This stops one outlier from crushing the field and keeps mid-table differences meaningful.

3.2 Diversification — 25% of the score

What it is: two measurements against the game's own 10-factor market model. (1)

ORTHOGONALITY: how many genuinely independent directions did the team's bets span?

Buying TCS and INFY is one tech bet made twice — the maths sees through the different ticker names. (2) NEUTRALITY: how much of the team's exposure was hedged — deliberately offset with opposite-signed exposures — the way a market-neutral fund runs its book.

Why it matters: it rewards understanding the MARKET'S STRUCTURE rather than chasing one story. A team that hedges gives up some profit for lower risk — that is sophisticated play, and without this criterion it would score WORSE than an unhedged gambler. Sitting in cash does not count as hedging (the maths ignores un-invested time).

Why 25: it is a craft signal, not the objective. High enough that a one-sector team cannot win on luck alone; low enough that diversification without profit doesn't win either.

3.3 Hit rate — 30% of the score

What it is: of the team's completed trades (buy → later sell, matched first-in-first-out; positions still open at the bell are closed at the final price), what fraction made money?

Why it matters: it is the cleanest read of whether the team's calls were RIGHT — did the news-to-stock reasoning work — independent of bet sizing. A team can be up big on one monster trade and wrong on everything else; hit rate reveals that.

Why 30: skill signal, second only to profit. Two honesty guards: tiny trades (under ½% of starting capital) don't count at all, and teams with very few trades have their rate shrunk toward 50% — one lucky trade is not a 100% hit rate.

3.4 The sliders

The 45 / 25 / 30 split is a defensible default, not a law of nature. The console has three sliders; judges can agree a different blend BEFORE the event and the board re-ranks live. Whatever is chosen must be announced to the players before the game starts — changing the objective after play is unfair.

Reading the board in 10 seconds

Rank and total /100 answer "who is winning". The three bars answer "how": a team high on Return but near-zero on Diversification made one concentrated bet that worked; a team mid on Return but high on the other two traded like a professional. Click the row — the radar (spread of bets), the equity curve (path, not just endpoint) and the blotter (every trade, HIT or MISS) are the evidence.

Part 4 — The full maths (the glue)

This part is the exact specification of every calculation. The code in Part 5 implements it function-for-function. Notation: i = a stock, k = a factor ($k = 1 \dots 10$), t = a tick, j = an order, K_0 = starting capital (1,000,000), $\beta[i]$ = stock i 's 10-vector of factor sensitivities parsed from `game_data.json`'s universe research strings, $P[i,t]$ = the canonical price of stock i at tick t from `game_data.json` rows.

All sums run over the scoring scope (both sessions, or one) selected in the console.

4.0 Replay — the common preprocessing step

From the team's chronological order list, rebuild the whole account tick by tick:

$$\text{cash}[t], \text{shares}[i,t] \rightarrow V[t] = \text{cash}[t] + \sum_i \text{shares}[i,t] \cdot P[i,t]$$

- **Validation** — $\max_t |V[t] - V_{\text{submitted}}[t]| / K_0 < 10^{-6}$, else flag the bundle (cross-check only; the replayed values are always the ones scored).
- **Session boundary rule** — sessions are scored as separate books. At the session-1 $\rightarrow$ session-2 boundary: force-close every open lot at the last session-1 price (these become mark-to-market round-trips), reset cash to K_0 , clear holdings. Scope "Both" then means: return is the AVERAGE of the two session returns; trips, bet vectors and exposure sums simply pool across both sessions.
- **Orders are trusted as events, not as maths** — ticker/side/qty/tick come from the client, but the execution price used in replay is the canonical $P[\text{ticker}, \text{tick}]$, NOT the price field in the order (which is kept and compared as another cross-check).

Client bug this rule exposes (must fix in app.js)

The v5 spec says session 2 is a fresh scoreboard, but the current `app.js` does NOT reset cash/holdings when the session flips — prices snap back to their start values while positions persist, creating phantom P&L for anyone holding across the boundary. Part 6.1 includes the two-line reset as a required client change.

4.1 Subscore 1 — Return S_R

$$r = (V[T] - K_0) / K_0 \quad (T = \text{last tick in scope}; \\ \text{scope Both: } r = (r_{s1} + r_{s2})/2)$$

$$S_R = 1 / (1 + e^{-(r - r_0)/\lambda}) \quad \text{with } r_0 = 0.04, \lambda = 0.075$$

- **r_0 (midpoint)** — the return that scores exactly 0.5. +4%: "par" money for a 40-minute game sits mid-scale.

- **λ (spread)** — saturation speed. At $\lambda = 0.075$: $r = -10\% \rightarrow 0.13$, $0\% \rightarrow 0.37$, $+12\% \rightarrow 0.74$, $+20\% \rightarrow 0.89$.
- **Why a sigmoid** — monotonic (more return never scores less), bounded (one outlier cannot flatten the field), and both knobs have plain meanings (Part 7).

4.2 Subscore 2a — Orthogonality via effective rank

Each order j (ticker i_j , side $s_j = +1$ BUY / -1 SELL, quantity q_j , canonical price p_j) becomes a BET VECTOR in factor space:

$$u_j = s_j \cdot (q_j \cdot p_j / K_o) \cdot \beta[i_j] \quad \in \mathbb{R}^{10}$$

Stack the u_j as rows of matrix B (N orders $\times 10$). With singular values $\sigma_1 \geq \dots \geq \sigma_{10}$ of B , the PARTICIPATION RATIO counts how many independent directions the bets truly span:

$$R_{\text{eff}} = (\sum_m \sigma_m^2)^2 / \sum_m \sigma_m^4 \quad \in [1, 10]$$

$$D_{\text{rank}} = \min(R_{\text{eff}} / R_{\text{cap}}, 1) \quad \text{with } R_{\text{cap}} = 5$$

Implementation note — no SVD library needed. Let $G = B^T B$ (a 10×10 symmetric matrix). Then $\sum \sigma^2 = \text{trace}(G)$ and $\sum \sigma^4 = \text{trace}(G^2) = \sum_{k,l} G[k,l]^2$, so:

$$R_{\text{eff}} = \text{trace}(G)^2 / \sum_{\{k,l\}} G[k,l]^2 \quad (G = B^T B, 10 \times 10)$$

- **What R_{eff} means** — trades all loading one factor combination put all variance in $\sigma_1 \rightarrow R_{\text{eff}} \approx 1$ no matter how many trades. Bets spread evenly over m orthogonal directions $\rightarrow R_{\text{eff}} \approx m$.
- **Size-weighting** — $q \cdot p / K_o$ scales each row by notional: token trades contribute negligible mass and cannot inflate rank.
- **Round-trips don't fake rank** — a SELL closing a BUY produces $-u$, collinear with $+u$; collinear rows add no new direction.
- **$R_{\text{cap}} = 5$** — full marks at 5 effective factors: with each stock loading 3–4 of the 10 factors, ~ 5 is the realistic ceiling for a deliberate multi-factor book. Requiring 10 would reward noise.

4.3 Subscore 2b — Neutrality (the hedging reward)

From the replayed holdings, at every tick compute position weights and per-factor NET vs GROSS exposure:

$$w[i,t] = \text{shares}[i,t] \cdot P[i,t] / V[t] \quad (\text{long-only} \Rightarrow w \geq 0)$$

$$E[k,t] = \sum_i w[i,t] \cdot \beta[i,k] \quad (\text{NET} - \text{opposite signs cancel})$$

$$G[k,t] = \sum_i |w[i,t] \cdot \beta[i,k]| \quad (\text{GROSS} - \text{nothing cancels})$$

$$v = 1 - (\sum_t \sum_k |E[k,t]|) / (\sum_t \sum_k G[k,t]) \quad \in [0, 1]$$

$$D_{\text{neut}} = \min(v / N_{\text{cap}}, 1) \quad \text{with } N_{\text{cap}} = 0.5$$

- **What v means** — the fraction of the book's gross factor exposure CANCELLED by offsetting positions. One-directional book: $|E| = G$ every tick $\Rightarrow v = 0$. Substantially hedged book $\Rightarrow v \rightarrow 1$.
- **Cash cannot game it** — un-invested ticks contribute 0 to numerator and denominator alike; an all-cash team has $\sum G = 0$, defined $v = 0$. You cannot be "neutral" by not playing.
- **Gross-weighting over time** — a team hedged only for the final 30 seconds gets credit only for those ticks, in proportion to book actually run.
- **N_cap = 0.5** — the game is long-only, so hedging is only possible by pairing opposite-signed betas (e.g. IT names carry RatesRupree -0.20 while banks carry it positive). With the current universe the best deliberately hedged long-only book reaches $v \approx 0.4\text{--}0.5$; full marks at 0.5 keeps the target reachable. Recompute if betas change (Part 7).

4.4 Combining 2a + 2b

$$S_D = \omega_{\text{rank}} \cdot D_{\text{rank}} + \omega_{\text{neut}} \cdot D_{\text{neut}} \quad \text{with } \omega_{\text{rank}} = 0.6, \omega_{\text{neut}} = 0.4$$

Orthogonality is the primary reading of "many independent bets"; neutrality is the deliberate-hedger bonus. A team hedging away profit loses on S_R and gains here — by design, per the project owner's requirement that market-neutral construction be rewarded, not punished.

4.5 Subscore 3 — Hit rate S_H

ROUND-TRIP CONSTRUCTION (FIFO). Per ticker, BUYs create lots; each SELL consumes the OLDEST open lots first (one SELL may close several lots $\rightarrow$ several trips; one lot may be closed by several SELLS). Each matched pairing of q shares, entry p_{in} (canonical price at the buy tick), exit p_{out} (canonical price at the sell tick) is one TRIP:

$$\begin{aligned} \text{trip return } p &= (p_{\text{out}} - p_{\text{in}}) / p_{\text{in}} \\ \text{trip notional } n &= q \cdot p_{\text{in}} \end{aligned}$$

Lots still open at the end of scope (or at a session boundary) are force-closed at the last canonical price — mark-to-market trips; an unrealised loss is still a miss.

$$\text{Qualifying trips: } n \geq \theta \cdot K_0 \quad \text{with } \theta = 0.005 \text{ (\% of capital)}$$

```
hits = #{ qualifying trips with  $p > \theta$  }
n_tr = #{ qualifying trips }
```

```
S_H = ( hits +  $\kappa/2$  ) / ( n_tr +  $\kappa$  )           with  $\kappa = 2$  (shrink
                                                toward 0.5)
```

- **θ (de-minimis)** — trips under ½% of starting capital count for nothing, good or bad — 1-share confetti cannot pad the numerator.
- **κ (shrinkage)** — Laplace-style pull toward 50%. One winning trip scores $(1+1)/(1+2) = 0.67$, not 1.00; a 12-of-16 grinder scores $(12+1)/(16+2) = 0.72$ and correctly outranks the one-shot wonder. The shrink vanishes as n_{tr} grows.
- **Zero qualifying trips** — define $S_H = 0$ and flag "insufficient activity" (the shrinkage formula alone would hand out an unearned 0.5).

4.6 The total

```
Score = 100 · ( w_R · S_R + w_D · S_D + w_H · S_H ) / ( w_R + w_D + w_H )
```

Judge-set weights default to $w_R = 45$, $w_D = 25$, $w_H = 30$. Subscores are computed once per data refresh; a slider move re-runs only this line — which is why re-ranking is instant.

4.7 Worked micro-examples

Orthogonality

Factor space reduced to (GlobalTech, CrudeOil). $\beta(\text{TCS}) = (0.85, 0)$, $\beta(\text{INFY}) = (0.80, 0)$, $\beta(\text{ONGC}) = (0, 0.90)$.

- **Team Tech** buys ₹200k TCS and ₹200k INFY: $u_1 = 0.2 \cdot (0.85, 0) = (0.17, 0)$, $u_2 = (0.16, 0)$. Rows parallel $\rightarrow$ all variance in $\sigma_1 \rightarrow R_{\text{eff}} = 1.0$. Two stocks, ONE bet.
- **Team Spread** buys ₹200k TCS and ₹200k ONGC: $u_1 = (0.17, 0)$, $u_2 = (0, 0.18)$. Orthogonal rows $\rightarrow R_{\text{eff}} = (0.0289 + 0.0324)^2 / (0.0289^2 + 0.0324^2) \approx 1.99$. Two stocks, TWO bets.

Neutrality

Suppose HDFCBANK carries RatesRupee $\beta = +0.60$ and TCS carries -0.20 (only factor shown), and a team holds weights $w = 0.25$ in each for the whole game. Net $E = 0.25 \cdot 0.60 + 0.25 \cdot (-0.20) = +0.10$; gross $G = 0.15 + 0.05 = 0.20$. On this factor $v = 1 - 0.10/0.20 = 0.5$: half the rates exposure is hedged away. The real calculation does this across all 10 factors and all ticks at once.

Hit-rate shrinkage

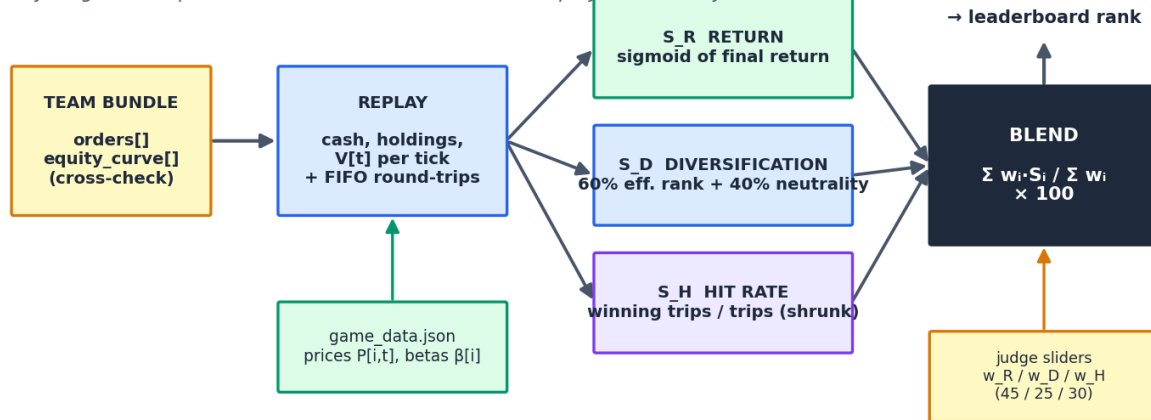
Team A: 1 trip, won it $\rightarrow (1+1)/(1+2) = 0.67$. Team B: 16 trips, won 12 $\rightarrow (12+1)/(16+2) = 0.72$. B outranks A despite A's "perfect" record — by design.

Part 5 — The code (developer guide)

Four files. scoring.js is pure maths (no DOM, no network) so it can be unit-tested and audited alone; admin.js is wiring and drawing; the app.js changes are recording only.

The scoring pipeline — one team, orders in, score out

Everything is recomputed from raw orders. The submitted equity curve is only a cross-check.



Subscores are computed ONCE per data refresh; moving a slider only re-runs the blend — which is why re-ranking is instant.

Figure. One team through the pipeline. Subscores cache per team; sliders only re-run the blend.

File	What it is	Depends on
firebase-glue.js	Init + two helpers: pushOrder(), watchGame(). ~40 lines. Shared by game client and console.	Firebase CDN SDK
app.js (edit)	Record + stream orders, equity snapshot per tick, localStorage mirror, bundle export, session-boundary reset fix.	firebase-glue.js
scoring.js (new)	parseBetas, replay, buildTrips, scoreReturn, effectiveRank, neutrality, scoreHitRate, blend. Implements Part 4 1:1.	nothing
admin.html/js (new)	Judge console page: subscribe → throttle → score → render (leaderboard, sliders, drill-down charts, CSV export).	both of the above

5.1 Reading the factor betas out of game_data.json

The betas live as text in each universe entry ("... sensitivities: GlobalTech +0.85, GlobalRisk +0.35, RatesRupee -0.20 ..."). Parse once at load:

```
const FACTORS = ["CrudeOil", "RatesRupee", "GlobalTech", "DomPolicy", "Metals",
                 "ConsDemand", "GlobalRisk", "Credit", "EnergyTx", "SupplyChain"];

function parseBetas(gameData) { // → { TCS: Float64Array(10), ... }
  const betas = {};
  for (const u of gameData.universe) {
    const v = new Float64Array(10);
```

```

const m = u.research.match(/sensitivities:\s*(.+)/);
if (m) for (const part of m[1].split(",")) {
  const pm = part.trim().match(/(\w+)\s+([+-][\d.]+)/);
  if (pm) v[FACTORS.indexOf(pm[1])] = parseFloat(pm[2]);
}
betas[u.ticker] = v;
}
return betas;
}

```

5.2 Replay (Part 4.0)

```

// orders: [{tick, ticker, side, qty}] sorted by tick. Prices ALWAYS canonical.
function replay(orders, gameData, scope) { // scope: 0, 1, or "both"
  const rows = gameData.rows; // 240 rows
  const K0 = 1_000_000;
  let cash = K0, shares = {}; // ticker → int
  const V = [], lots = {}, trips = []; // lots: ticker → [{qty,
tick}]
  let oi = 0;
  for (let t = 0; t < rows.length; t++) {
    const row = rows[t];
    if (t > 0 && rows[t-1].session !== row.session) { // session boundary:
      forceCloseAll(lots, shares, rows[t-1], trips); // MTM-close open lots
(4.5)
      cash = K0; shares = {}; // fresh book (4.0)
    }
    while (oi < orders.length && orders[oi].tick === t) {
      const o = orders[oi++], px = row.prices[o.ticker];
      if (o.side === "BUY") { cash -= px*o.qty; addLot(lots, o, t);
        shares[o.ticker] = (shares[o.ticker]||0) + o.qty;
      }
      else { cash += px*o.qty;
        fifoClose(lots, o, t, row, rows, trips); // 4.5
        shares[o.ticker] -= o.qty; }
    }
    let pos = 0;
    for (const tk in shares) pos += shares[tk] * row.prices[tk];
    V.push(cash + pos);
  }
  forceCloseAll(lots, shares, rows[rows.length-1], trips); // end-of-game MTM
  return { V, trips /* {tickIn, tickOut, ticker, qty, pIn, pOut, forced} */ };
}

```

fifoClose pops the oldest lots of that ticker until the sell quantity is filled, emitting one trip per (lot, sell) pairing; forceCloseAll emits a trip per remaining lot at the given row's price with forced = true. Rejecting impossible orders (sell > held, buy > cash at canonical price) also happens here: an impossible order flags the whole bundle rather than being silently skipped.

5.3 The three subscores

```

function scoreReturn(V, K0, r0 = 0.04, lam = 0.075) { // Part 4.1
  const r = (V[V.length-1] - K0) / K0; // per session; average for scope both
  return { r, sR: 1 / (1 + Math.exp(-(r - r0) / lam)) };
}

```

```

function effectiveRank(orders, betas, rowsByTick, K0, Rcap = 5) { // Part 4.2
  const G = Array.from({length:10}, () => new Float64Array(10)); // G = BTB
  for (const o of orders) {
    const w = (o.side === "BUY" ? 1 : -1) *
      o.qty * rowsByTick[o.tick].prices[o.ticker] / K0;
    const b = betas[o.ticker];
    for (let k = 0; k < 10; k++) for (let l = 0; l < 10; l++)
      G[k][l] += (w*b[k]) * (w*b[l]); // accumulate outer product uTu
  }
  let tr = 0, fro2 = 0;
  for (let k = 0; k < 10; k++) { tr += G[k][k];
    for (let l = 0; l < 10; l++) fro2 += G[k][l]*G[k][l]; }
  const Reff = fro2 > 0 ? (tr*tr) / fro2 : 0; // trace identity – no SVD
  return { Reff, dRank: Math.min(Reff / Rcap, 1) };
}

function neutrality(holdingsByTick, betas, Ncap = 0.5) { // Part 4.3
  let net = 0, gross = 0;
  for (const {w} of holdingsByTick) { // w: ticker → weight at tick t
    for (let k = 0; k < 10; k++) {
      let E = 0, Gk = 0;
      for (const tk in w) { const x = w[tk]*betas[tk][k]; E += x; Gk +=
Math.abs(x); }
      net += Math.abs(E); gross += Gk;
    }
  }
  const nu = gross > 0 ? 1 - net/gross : 0;
  return { nu, dNeut: Math.min(nu / Ncap, 1) };
}

function scoreHitRate(trips, K0, theta = 0.005, kappa = 2) { // Part 4.5
  const q = trips.filter(tp => tp.qty * tp.pIn >= theta * K0);
  const hits = q.filter(tp => tp.pOut > tp.pIn).length;
  if (q.length === 0) return { hits: 0, nTr: 0, sH: 0, flag: "insufficient" };
  return { hits, nTr: q.length, sH: (hits + kappa/2) / (q.length + kappa) };
}

```

5.4 The blend (and why sliders are free)

```

function blend(sub, w) { // Part 4.6
  const sD = 0.6 * sub.dRank + 0.4 * sub.dNeut; // Part 4.4
  return 100 * (w.R*sub.sR + w.D*sD + w.H*sub.sH) / (w.R + w.D + w.H);
}
// slider input → only blend() re-runs over cached subscores → resort → render

```

5.5 admin.js skeleton

```

const gameData = await (await fetch("game_data.json")).json();
const betas = parseBetas(gameData);
const cache = {}; // team → {sub, flags} (subscores)

watchGame(gameData.meta.seed, (teams) => { // Firebase onValue (Part 6.3)
  for (const [team, node] of Object.entries(teams)) markDirty(team, node);
}

```

```

});
setInterval(() => {
  for (const team of takeDirty()) {
    const orders = toSortedOrders(latest[team].orders);
    const rep = replay(orders, gameData, scope);
    cache[team] = { sub: subscores(rep, orders), flags: validate(rep,
latest[team]) };
  }
  render(cache, weights());
}, 2000);
// 2-second rescore throttle
// blend + sort + draw

```

Charts: the drill-down uses the same Chart.js already shipped with the game for the equity curve; the radar and donut are small hand-drawn SVGs (the mockup contains working reference implementations to lift directly).

5.6 Testing

- **Fixtures from Part 4.7 verbatim** — Team Tech $\rightarrow R_{\text{eff}} = 1.0$; Team Spread $\rightarrow R_{\text{eff}} \approx 1.99$; the neutrality example $\rightarrow v = 0.5$; shrinkage 1-for-1 $\rightarrow 0.67$.
- **FIFO edges** — partial fills (sell half a lot), one sell closing three lots, force-close at session boundary and at the bell, sell-more-than-held $\rightarrow$ bundle flagged.
- **Cash-gaming guard** — an all-cash team scores $v = 0$, not 1.
- **Cross-check** — corrupt one submitted equity point by ₹1 $\rightarrow$ row flags.
- **Determinism** — scoring the same orders twice gives identical output (no `Date.now()`, no `Math.random()` anywhere in `scoring.js`).

Part 6 — Wiring it together (developer guide)

6.1 Exact touchpoints in the existing app.js

Where (existing code)	Add
handleLogin()	After successful login: init firebase-glue with team name + gameData.meta.seed; restore any localStorage state for this team (crash recovery).
confirmOrder(), after the order is accepted	orderHistory.push already exists — also pushOrder({tick: currentTick, ticker, side, qty, price}) to Firebase AND mirror orderHistory to localStorage.
processTick(), end	Push cash + Σ shares-price into an equitySeries array UNCONDITIONALLY (today it only updates via the portfolio tab); mirror the last value + currentTick to localStorage.
processTick(), session-change branch	BUG FIX (Part 4.0): when session flips, force-liquidate holdings at the previous row's prices into cash, then reset cash = START_CAPITAL, holdings = {} — matching the "fresh scoreboard" spec.
Game-over path in startGame()'s interval	Serialize the results bundle (6.4) and (a) write it to Firebase under /bundles, (b) offer it as a file download — the offline fallback.

6.2 Firebase project setup (one-time, ~15 minutes)

1. console.firebase.google.com → Add project (no Analytics needed). The default Spark plan is free: no credit card, 1 GB stored, 100 simultaneous connections — a 10-team event uses under 1% of every limit.
2. Build → Realtime Database → Create database → start in locked mode.
3. Project settings → your apps → Web app → copy the config object into firebase-glue.js.
4. Paste the rules below into Realtime Database → Rules.

```
{
  "rules": {
    "games": {
      "$seed": {
        "teams": {
          "$team": {
            ".read": true,
            "orders": {
              "$oid": {
                // append-only: new orders may be created, never edited/deleted
                ".write": "!data.exists()",
                ".validate": "newData.hasChildren(['tick','ticker','side','qty'])
                            && newData.child('qty').isNumber()
                            && newData.child('qty').val() > 0
                            && (newData.child('side').val() === 'BUY'
                                || newData.child('side').val() === 'SELL')"
              }
            },
            "bundles": { ".write": true },
            "meta":      { ".write": true }
          }
        },
        "bundles": { ".write": true },
        "meta":      { ".write": true }
      }
    }
  }
}
```

```

    }
  }
}

```

Security stance (accepted risk, stated openly)

There is no auth: any client that knows the database URL could write under any team's path. For a supervised classroom event this is accepted: the rules make orders APPEND-ONLY and shape-validated, every order is timestamped, and the replay flags impossible orders (selling shares never bought, spending cash never held) — so fabricated histories are detectable, not silent. The cheap mitigations if wanted later: a per-team secret path segment handed out at login, or Firebase Anonymous Auth + per-uid paths. Do NOT invest in more than that for a game.

6.3 firebase-glue.js

```

import { initializeApp } from
  "https://www.gstatic.com/firebasejs/10.12.0/firebase-app.js";
import { getDatabase, ref, push, onValue, set }
  from "https://www.gstatic.com/firebasejs/10.12.0/firebase-database.js";

const app = initializeApp({ /* config from 6.2 step 3 */ });
const db = getDatabase(app);
let base = null; // set at login

export function initGlue(seed, team) { base = `games/${seed}/teams/${team}`; }

export function pushOrder(order) { // fire-and-forget; game never blocks on it
  if (base) push(ref(db, `${base}/orders`), { ...order, ts: Date.now() })
    .catch(() => { /* offline: localStorage still has it */ });
}

export function watchGame(seed, cb) { // console side: live feed of all teams
  onValue(ref(db, `games/${seed}/teams`), snap => cb(snap.val() || {}));
}

```

The game must NEVER block on Firebase: pushOrder is fire-and-forget, and localStorage is written synchronously first. If the venue wifi dies mid-game, play continues untouched and the export file carries the full record.

6.4 The results bundle (offline fallback format)

```

{
  "schema_version": 2,
  "team": "alpha",
  "game_seed": 25345,
  "orders": [ { "tick": 37, "ticker": "TCS", "side": "BUY",
    "qty": 10, "price": 3855.20, "ts": 1751871234567 } ],
  "equity_curve": [1000000.0, 1000212.4, "..."],
  "final_cash": 512340.55
}

```

- **orders** is the authoritative field — everything is recomputed from it (with canonical prices; the price field here is itself only a cross-check).
- **equity_curve / final_cash** cross-check only (Part 4.0); disagreement flags, never silently wins.
- **game_seed** must equal the seed in the judges' game_data.json or the bundle is from a different game and is rejected loudly.

6.5 Event-day runbook

5. Before: generate the game (market_engine.py), deploy game_data.json to BOTH the game site and the console, agree weights with the judges, announce them to players.
6. Dry run: one browser plays a few trades; confirm the row appears live on the console and the cross-check is green.
7. During: judges watch the live board. Any flagged row → investigate via the drill-down blotter BEFORE the reveal, not after.
8. End: sanity-glance that every team's bundle also landed under /bundles (or collect export files from any machine that went offline), lock the weights, export the CSV for the record.

Part 7 — Tuning the knobs (owner)

Every constant in Part 4, in one table. Change only before an event; re-announce if players were told the old values.

Knob	Default	Meaning	When to change
r_o	0.04	Return that scores exactly 50	Raise for longer/easier games; roughly the median return you expect
λ	0.075	Return score spread	Smaller = more separation among mid-field teams
R_cap	5	Effective factors for full orthogonality marks	$\approx$ half the factor count; recompute if universe/factors change
N_cap	0.5	Neutrality v for full hedging marks	Recompute whenever betas change: simulate the best long-only hedge in the new universe and set N_cap near it
$\omega_{\text{rank}} / \omega_{\text{neut}}$	0.6 / 0.4	Split inside diversification	Shift toward ω_{neut} to emphasise market-neutral craft
θ	0.005	De-minimis trip size ($\times K_o$)	Raise if teams spam small trades anyway
κ	2	Hit-rate shrink strength	Raise if games produce very few trips per team
$w_R / w_D / w_H$	45 / 25 / 30	Default blend weights	The judges' values call — sliders at event time
Rescore throttle	2 s	Console recompute cadence	Raise if many teams / slow judge laptop

Calibration ritual (same spirit as the engine's Part 7)

Before an event, run 4 synthetic teams through scoring.js: an all-in-one-stock team, a balanced multi-factor team, a deliberate hedger, and a spray-and-pray bot. Check the ranking matches your intuition at default weights; if not, tune the knobs above — never the equations. Keep the four fixtures as permanent unit tests.

Free-tier reality check

Firebase Spark plan limits vs a 10-team, 240-tick event: connections 100 vs ~ 15 used; storage 1 GB vs < 1 MB used; downloads 10 GB/month vs a few MB. Cost: ₹0. The plan requires no credit card, so there is no accidental-billing risk.

End of v2. Supersedes v1 (which covered the console front end only). Adds: Firebase backend + live scoring, client recording & export, session-boundary rule + client bug fix, canonical-price replay, trace-identity R_{eff} implementation, full code guides, security stance, and the event-day runbook.